



Unified Generative AI Models via CommandLine Interface

GROUP - 6

M D T A N V E E R H O S S A I N N I H A L

A R A F T A H S A N P A V E L

I S M O T A R A E M U



About Project

Over the semester, our team has been working on a system that unifies multiple Generative AI models under a single, easy-to-use command-line interface.

In the world of AI, different models—like GPT-2 for text, DCGAN for images, or WaveGAN for audio—often require entirely different setups, scripts, and workflows. That makes experimenting across domains time-consuming and error-prone.

Our project solves this by creating one framework that can train and run a variety of models, all using the same command structure. Whether you want to fine-tune GPT-2 on a custom dataset, generate images with CGAN or CVAE, or synthesize audio with WaveGAN, you can do it with a single, consistent command.

The framework is modular, so adding new models or datasets is straightforward, and it automatically handles dataset preparation, logging, and result storage. This makes it not just a learning project, but a practical tool for anyone working with generative AI.

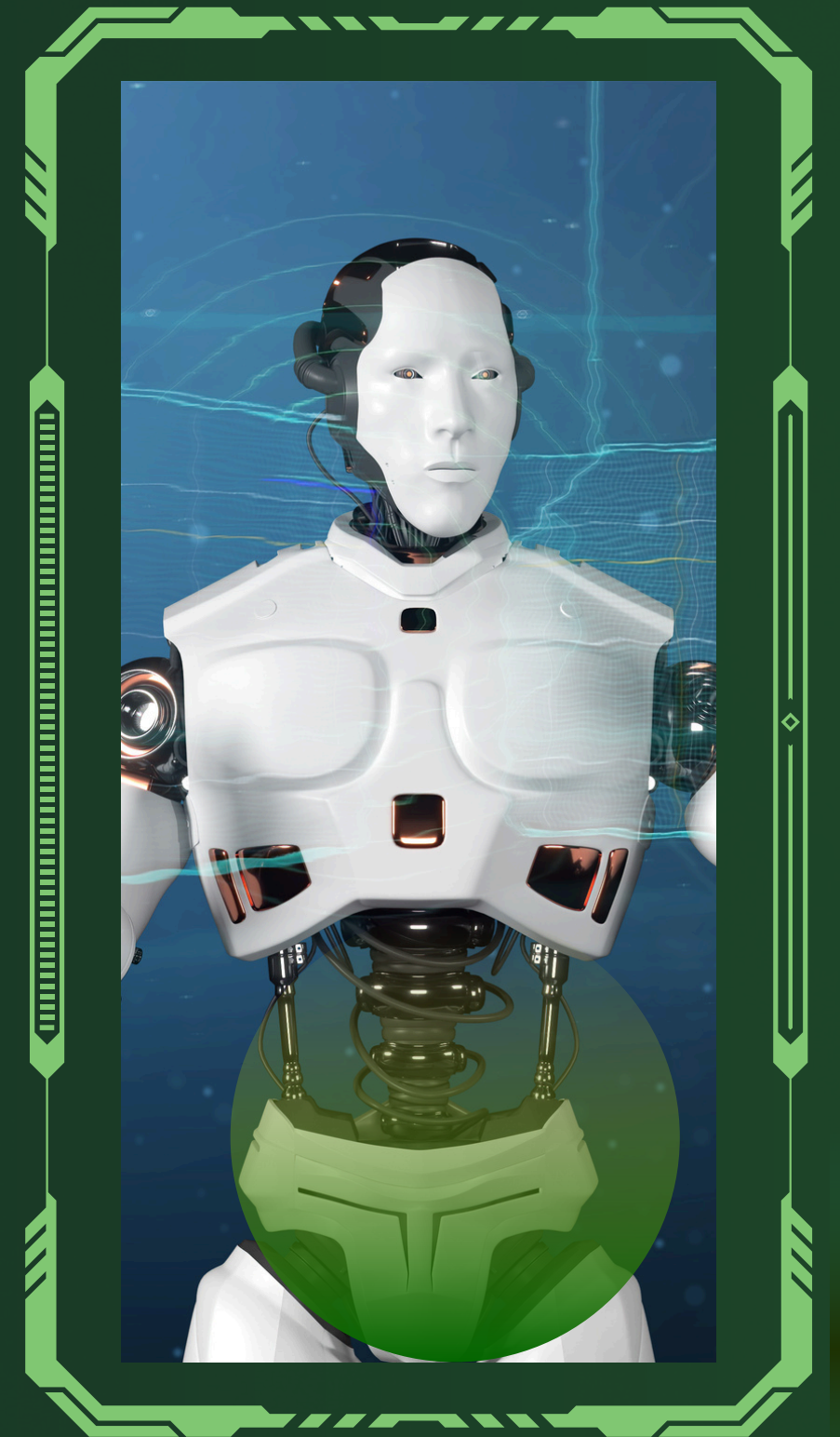


Motivation

In AI research and development, each generative model often comes with its own unique training script, dataset format, hyperparameter settings, and even dependencies. If you want to experiment with multiple models—say, GPT-2 for text and DCGAN for images—you typically have to set up two completely different environments, learn two different workflows, and maintain two separate codebases.

This fragmentation slows down experimentation, makes reproducibility harder, and increases the risk of human error. Our project eliminates this barrier by standardizing the workflow. Instead of juggling separate scripts and methods, every model—regardless of whether it works on text, images, or audio—is accessible through the exact same command-line structure. For example, training GPT-2 and training a Conditional GAN both follow the same pattern: you specify the dataset, the model name, and any additional options, and the framework handles the rest—dataset preparation, model initialization, training, and saving outputs in a consistent format.

This unified approach not only saves time but also makes the system modular and extensible, allowing new models to be added seamlessly without rewriting the whole training pipeline.





Project Goals

- PROVIDE ONE CLI(COMMAND LINE INTERFACE) COMMAND FOR ALL SUPPORTED MODELS.
- MODULAR ARCHITECTURE — EASY TO ADD NEW MODELS.
- AUTOMATICALLY HANDLE DATASET PREPARATION.
- SAVE OUTPUTS, LOGS, AND CHECKPOINTS IN A CONSISTENT FORMAT.



Directory Structure

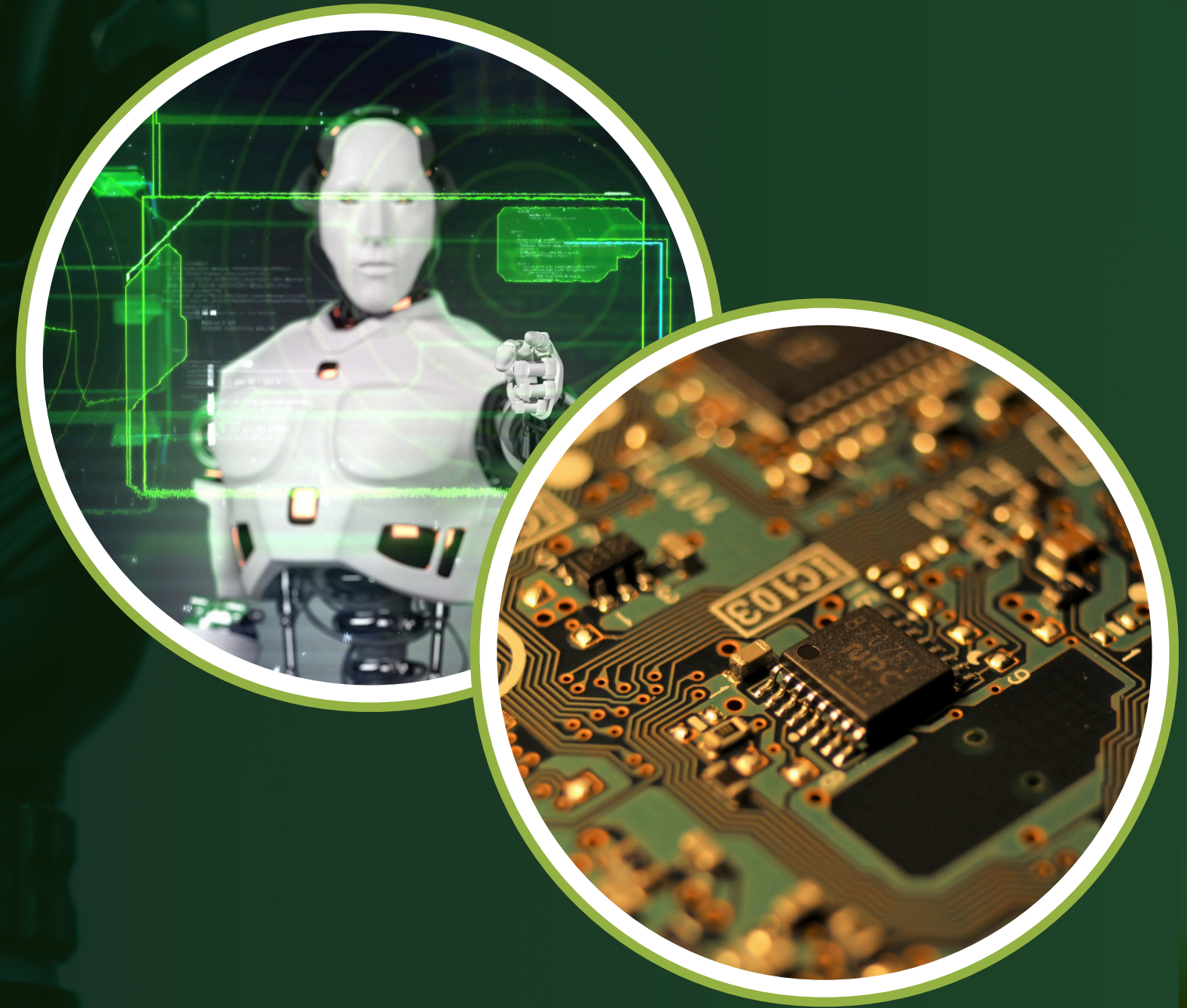
codeB/ → Bash scripts to run models
codeP/ → Python source (datasets, models, runner, main)
DATA/ → Organized datasets (training/testing)
RESULTS/ → Saved results, logs, models

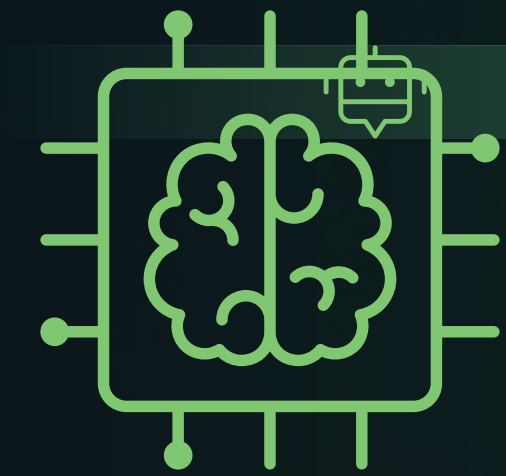
```
python codeP/main.py --dataset dataset_name --model model_name --otherOptions options
```



Supported Models

- GPT-2 – TEXT GENERATION (TXT, PDF, DOCS)
- CGAN – CONDITIONAL IMAGE GENERATION (MNIST)
- CVAE – LATENT SPACE CONTROLLED GENERATION (MNIST)
- WAVEGAN – AUDIO WAVEFORM SYNTHESIS (ESC-50)
- DCGAN – IMAGE GENERATION (CIFAR-10)





GPT-2 Module

- TRANSFORMER-BASED LANGUAGE MODEL BY OPENAI.
- SUPPORTS .TXT, .PDF, .DOCX TRAINING DATA.
- PREPROCESSING → TOKENIZATION → FINE-TUNING WITH HUGGING FACE API.
- SUPPORTS MULTIPLE PROMPT FILES AND BATCH GENERATION.
- OUTPUTS GENERATED TEXT FILES TO RESULTS/GPT2_OUTPUT/.

```
# gpt2.py - core training setup
from transformers import GPT2Tokenizer, GPT2LMHeadModel, Trainer, TrainingArgument

tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

model = GPT2LMHeadModel.from_pretrained("gpt2")

training_args = TrainingArguments(
    output_dir="./DATA/output",
    per_device_train_batch_size=2,
    num_train_epochs=30.0,
    save_steps=500,
    logging_steps=100,
    save_total_limit=2,
    prediction_loss_only=True,
    remove_unused_columns=False,
)

trainer = Trainer(
    model=model,
    args=training_args,
    data_collator=data_collator,
    train_dataset=train_dataset,
)

trainer.train()
trainer.save_model()
```

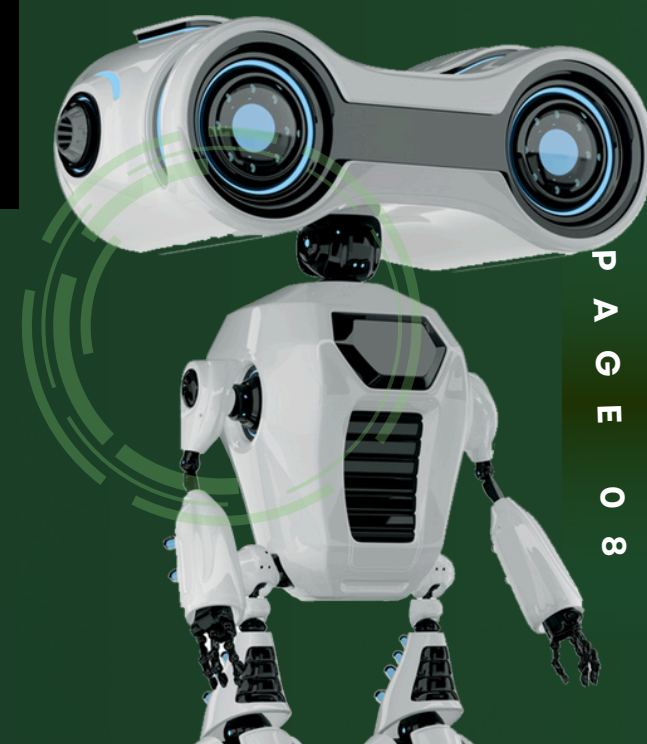
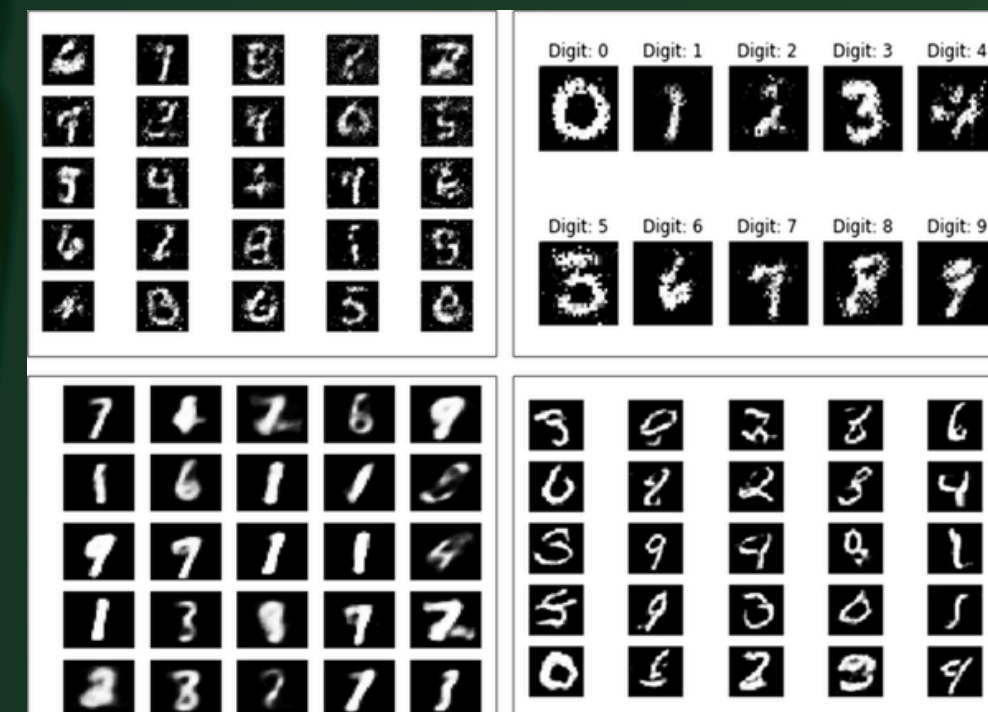
```
RESULT 001
Prompt: What is North South University?
Generated on: 2025-07-30 02:05:19
Max length: 100
-----

Generated Answer:
What is North South University?
North South University (NSU) is a private research
university located in Dhaka, Bangladesh. It was
established on 12 January 2009 as a result of the
Foundation for Promotion of Education and Research (
FPER), which provided economic and administrative
stability in Bangladesh. The university is widely
regarded as one of the top private universities in the
country, known for its academic excellence and modern
campus facilities. The university is widely regarded as
one of the top private universities
```



CGAN Module(Conditional Image Generation)

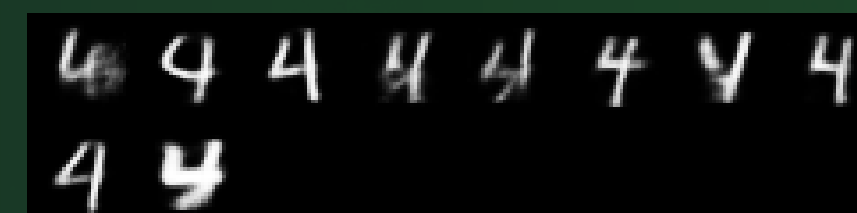
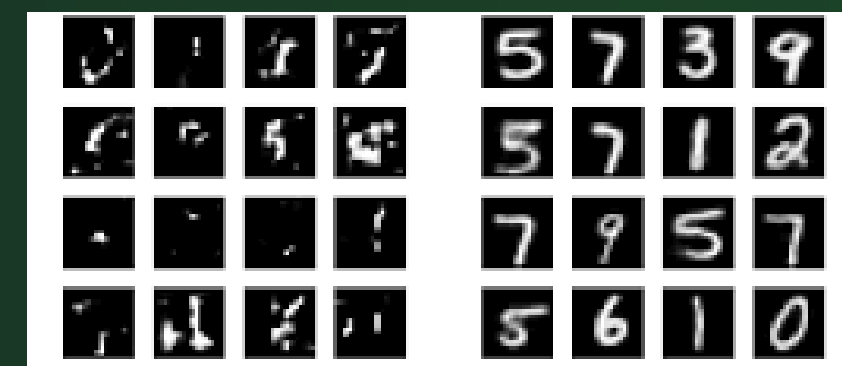
- GAN VARIANT WITH LABEL-CONDITIONED GENERATION.
- GENERATES CLASS-SPECIFIC IMAGES (E.G., MNIST DIGITS).
- MNIST DIGITS CONDITIONED ON LABELS.
- GENERATOR + DISCRIMINATOR WITH LABEL EMBEDDINGS.
- USEFUL FOR TARGETED SYNTHETIC DATASETS.





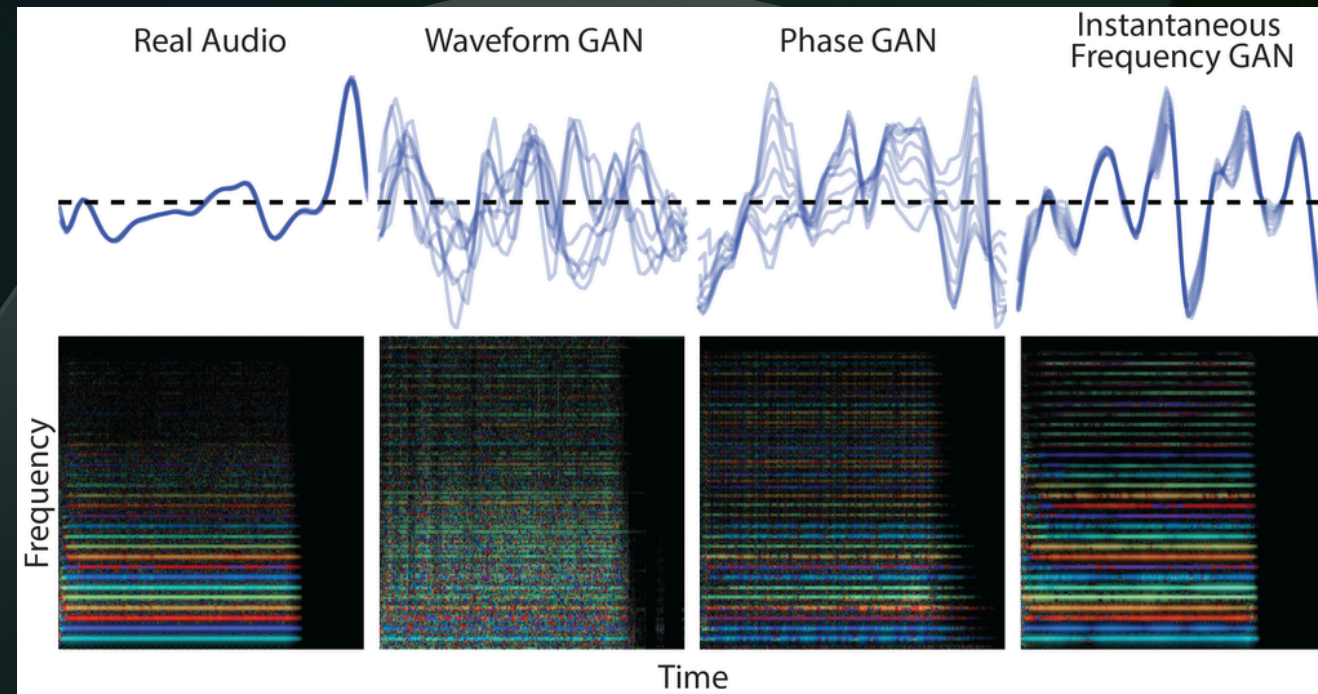
CVAE (Conditional Variational Autoencoder)

- ENCODER-DECODER ARCHITECTURE WITH PROBABILISTIC LATENT SPACE.
- LEARNS PROBABILISTIC LATENT SPACE.
- GENERATES PER-CLASS SAMPLES, RECONSTRUCTIONS, AND INTERPOLATIONS.
- OUTPUTS TO RESULTS/CVAE/.





WaveGAN Module

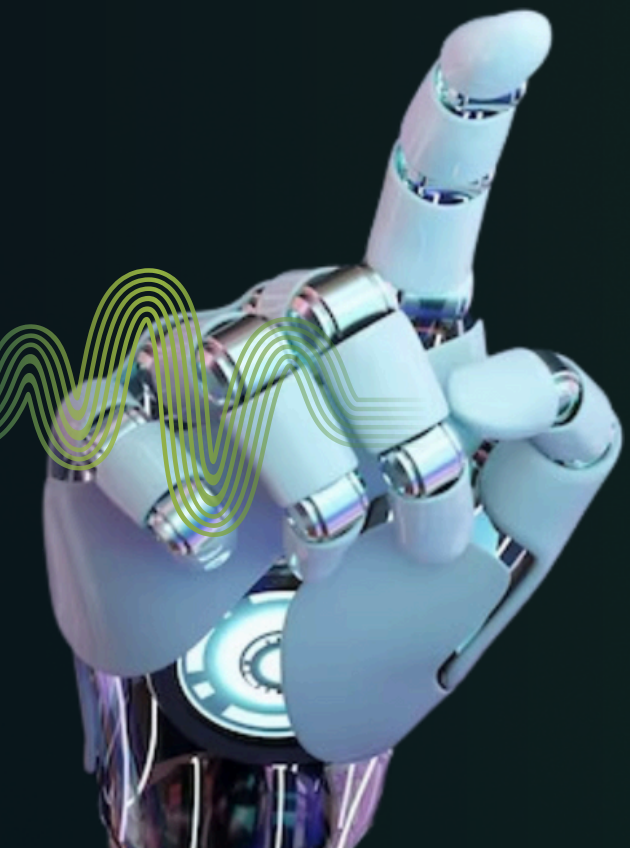


- 1D CONVTRANSPOSE LAYERS FOR UPSAMPLING NOISE TO WAVEFORM

- TRAINS ON ESC-50 ENVIRONMENTAL SOUNDS.

- GENERATES .WAV FILES AT INTERVALS.

- OUTPUTS SAVED IN GENERATED_SAMPLES/ AND FINAL_SAMPLES/.





DCGAN (Deep Convolutional GAN)

- DEEP CONVOLUTIONAL GAN ARCHITECTURE FOR REALISTIC IMAGES.
- GENERATOR: CONVTRANSPOSE2D LAYERS, DISCRIMINATOR: CONV2D LAYERS.
- TRAINS ON CIFAR-10, RESIZED TO 64×64.
- PERIODICALLY SAVES GENERATED IMAGE GRIDS (SAMPLE_EPOCH_XX.PNG).
- GOOD FOR LEARNING DEEP CONVOLUTIONAL GENERATION.

```
# dcgan.py - Generator & Discriminator
class Generator(nn.Module):
    def __init__(self, nz=100, ngf=64, nc=3):
        super().__init__()
        self.main = nn.Sequential(
            nn.ConvTranspose2d(nz, ngf*8, 4, 1, 0, bias=False),
            nn.BatchNorm2d(ngf*8), nn.ReLU(True),
            nn.ConvTranspose2d(ngf*8, ngf*4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf*4), nn.ReLU(True),
            nn.ConvTranspose2d(ngf*4, ngf*2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf*2), nn.ReLU(True),
            nn.ConvTranspose2d(ngf*2, ngf, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf), nn.ReLU(True),
            nn.ConvTranspose2d(ngf, nc, 4, 2, 1, bias=False),
            nn.Tanh()
        )
    def forward(self, z): return self.main(z)

class Discriminator(nn.Module):
    def __init__(self, nc=3, ndf=64):
        super().__init__()
        self.main = nn.Sequential(
            nn.Conv2d(nc, ndf, 4, 2, 1, bias=False), nn.LeakyReLU(0.2, True),
            nn.Conv2d(ndf, ndf*2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf*2), nn.LeakyReLU(0.2, True),
            nn.Conv2d(ndf*2, ndf*4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf*4), nn.LeakyReLU(0.2, True),
            nn.Conv2d(ndf*4, ndf*8, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf*8), nn.LeakyReLU(0.2, True),
            nn.Conv2d(ndf*8, 1, 4, 1, 0, bias=False), nn.Sigmoid()
        )
    def forward(self, x): return self.main(x).view(-1)
```





Technical Highlights

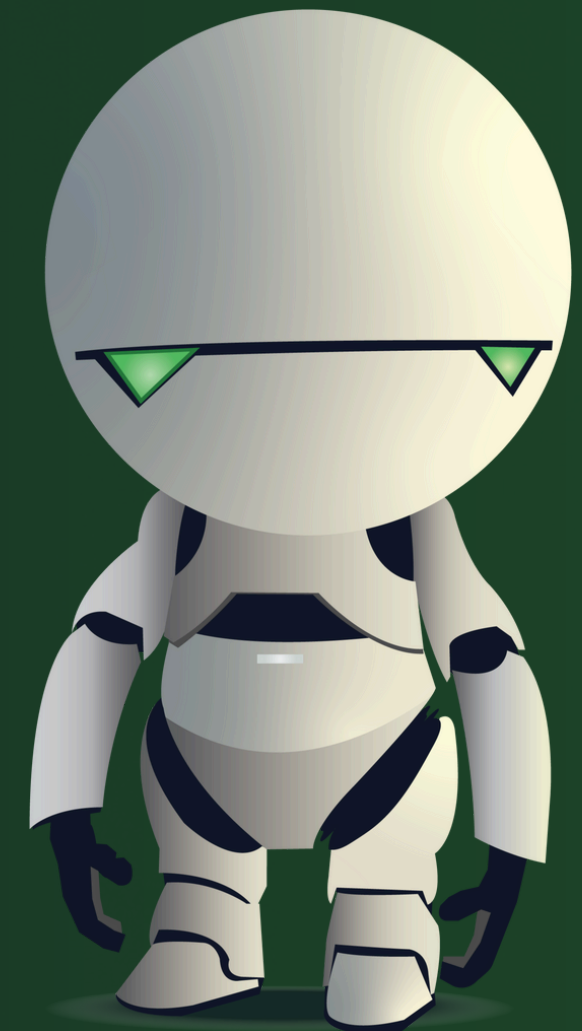
- MODULAR DATASET LOADERS IN CODEP/DATASETS/.
- CENTRAL RUNNER FOR TRAINING & LOGGING.
- AUTO GPU/CPU DETECTION.
- PRE-BUILT BASH SCRIPTS FOR REPRODUCIBILITY.





Results Summary

- GPT-2: PRODUCES COHERENT DOMAIN-SPECIFIC TEXT.
- CGAN & CVAE: CLEAR DIGIT GENERATION WITH LABEL CONTROL.
- WAVEGAN: GENERATES PLAUSIBLE ENVIRONMENTAL AUDIO.
- DCGAN: PRODUCES CIFAR-LIKE IMAGES AFTER FEW EPOCHS.





Strengths & Limitations

Strengths :

- UNIFIED INTERFACE
- EXTENSIBLE ARCHITECTURE
- SUPPORTS MULTIPLE MODALITIES

Limitations :

- LIMITED DATASETS AND LIMITED HARDWARE RESOURCES
- FIXED HYPERPARAMETERS IN CODE
- NO GUI FOR NON-TECHNICAL USERS



Future Work

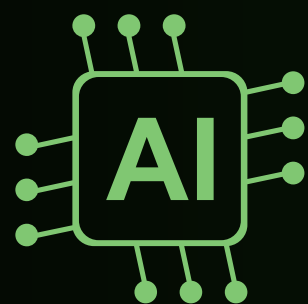
- ADD MORE STATE-OF-THE-ART MODELS (E.G., DIFFUSION MODELS).
- EXPAND DATASET SUPPORT.
- ADD CONFIG FILES FOR HYPERPARAMETER TUNING.



Conclusion



- DELIVERED A SINGLE FRAMEWORK TO RUN TEXT, IMAGE, AND AUDIO GENERATIVE MODELS FROM CLI.
- MODULAR, EXTENSIBLE, AND REPRODUCIBLE DESIGN.
- BRIDGES GAP BETWEEN EXPERIMENTATION AND DEPLOYMENT.



Thank
You!